

# SQL Parser From Scratch

## 1. Implement Lexer

|                   |      |
|-------------------|------|
| 1.1. Token Types  | [01] |
| 1.2. Token        | [01] |
| 1.3. KeywordTable | [02] |
| 1.4. SqlLexer     | [03] |
| 1.5. Comments     | [03] |

## 2. Grammer

|                             |      |
|-----------------------------|------|
| 2.1. Initial Grammer        | [04] |
| 2.2. SELECT Grammar         | [04] |
| 2.3. FROM Grammer           | [04] |
| 2.4. Expression Grammar     | [04] |
| 2.5. JOIN Grammer           | [05] |
| 2.6. WHERE / GROUP / HAVING | [05] |
| 2.7. ORDER / LIMIT / OFFSET | [05] |
| 2.8. Operator Precedence    | [05] |

## 3. AST

|                           |      |
|---------------------------|------|
| 3.1. SqlNode              | [06] |
| 3.2. SqlStatement         | [06] |
| 3.3. SelectStatement      | [06] |
| 3.4. InsertStatement      | [06] |
| 3.5. UpdateStatement      | [06] |
| 3.6. DeleteStatement      | [06] |
| 3.7. CreateTableStatement | [07] |
| 3.8. TABLE AST            | [07] |
| 3.9. JOIN AST             | [07] |
| 3.10. Column AST          | [07] |
| 3.11. Literals            | [07] |
| 3.12. BinaryExpression    | [08] |
| 3.13. Unary Expression    | [08] |
| 3.14. NULL / IN/ BETWEEN  | [08] |
| 3.15. CASE                | [08] |
| 3.16. Function Calls      | [08] |

# Project Structure

```
sql-parser/  
├── pom.xml  
├── README.md  
├── docs/  
│   ├── grammar.md  
│   ├── architecture.md  
│   └── dialects.md  
└── src/  
    ├── main/java/com/sqlparser/  
    │   ├── api/  
    │   ├── source/  
    │   ├── diagnostics/  
    │   ├── lexer/  
    │   ├── parser/  
    │   ├── ast/  
    │   ├── type/  
    │   ├── semantic/  
    │   ├── normalize/  
    │   ├── plan/  
    │   ├── optimizer/  
    │   ├── physical/  
    │   ├── executor/  
    │   ├── dialect/  
    │   ├── formatter/  
    │   └── cli/  
    └── test/java/com/sqlparser/  
        ├── lexer/  
        ├── parser/  
        ├── semantic/  
        ├── optimizer/  
        └── formatter/
```

```
├── plan  
│   ├── LogicalPlan.java  
│   ├── LogicalScan.java  
│   ├── LogicalFilter.java  
│   ├── LogicalProject.java  
│   ├── LogicalJoin.java  
│   ├── LogicalAggregate.java  
│   └── LogicalSort.java  
├── optimizer  
│   ├── OptimizationRule.java  
│   ├── PlanOptimizer.java  
│   ├── PredicatePushdown.java  
│   └── ProjectionPruning.java  
├── physical  
│   ├── PhysicalPlan.java  
│   ├── PhysicalScan.java  
│   ├── HashJoin.java  
│   ├── NestedLoopJoin.java  
│   └── SortOperator.java  
├── executor  
│   ├── Executor.java  
│   ├── PhysicalOperator.java  
│   └── Row.java  
└── cli  
    └── SqlCli.java
```

```
├── api  
│   ├── SqlParser.java  
│   ├── ParseResult.java  
│   └── ParserConfiguration.java  
├── source  
│   ├── SqlSource.java  
│   └── SourceLocation.java  
├── diagnostics  
│   ├── Diagnostic.java  
│   └── DiagnosticReporter.java  
├── lexer  
│   ├── SqlLexer.java  
│   ├── Token.java  
│   ├── TokenType.java  
│   └── KeywordTable.java  
├── parser  
│   ├── SqlParserImpl.java  
│   ├── SelectParser.java  
│   ├── ExpressionParser.java  
│   ├── DmlParser.java  
│   ├── DdlParser.java  
│   └── TokenStream.java  
├── ast  
│   ├── SqlNode  
│   ├── SqlStatement.java  
│   ├── SelectStatement.java  
│   ├── InsertStatement.java  
│   ├── UpdateStatement.java  
│   ├── DeleteStatement.java  
│   ├── CreateTableStatement.java  
│   ├── Expression.java  
│   ├── BinaryExpression.java  
│   ├── FunctionCallExpression.java  
│   ├── SubqueryExpression.java  
│   ├── Join.java  
│   └── ...  
├── type  
│   ├── SqlType.java  
│   ├── IntType.java  
│   ├── DecimalType.java  
│   ├── VarcharType.java  
│   └── ...  
├── semantic  
│   ├── SemanticAnalyzer.java  
│   ├── Scope.java  
│   ├── Catalog.java  
│   ├── TableMetadata.java  
│   ├── ColumnMetadata.java  
│   └── FunctionRegistry.java  
└── formatter  
    └── SqlFormatter.java
```

# 1. Implement lexer

## Token Types

|                                                                                                                                                                                                                                                                                          |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |                                                                                                                                                                                                                                                                                    |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p><b>Source Location:</b></p> <p>Every token and AST node should carry source information.</p> <pre>public record SourceLocation(     int offset,     int line,     int column ) {}</pre> <p>This enables:</p> <p>query.sql:4:18</p> <p>SELECT *<br/>FROM users<br/>WHERE age &gt;=</p> | <pre>public enum TokenType {     SELECT, FROM, WHERE,      INSERT, INTO, VALUES,     UPDATE, SET, DELETE,      CREATE, ALTER, DROP, TABLE,      AS, DISTINCT,     GROUP, BY, HAVING,     ORDER, ASC, DESC,     LIMIT, OFFSET,      JOIN, INNER, LEFT, RIGHT,     FULL, OUTER, CROSS, ON,      UNION, ALL, INTERSECT, EXCEPT,      WITH, RECURSIVE,      AND, OR, NOT,     IN, IS, NULL, BETWEEN, LIKE,     EXISTS,      CASE, WHEN, THEN, ELSE, END,      INT, BIGINT, DECIMAL,     VARCHAR, CHAR, BOOLEAN,     DATE, TIMESTAMP,</pre> | <pre>INTEGER_LITERAL, DECIMAL_LITERAL, STRING_LITERAL, BOOLEAN_LITERAL,  IDENTIFIER, QUOTED_IDENTIFIER,  PLUS, MINUS, STAR, SLASH, PERCENT,  EQUAL, NOT_EQUAL, LESS, LESS_EQUAL, GREATER, GREATER_EQUAL,  LEFT_PAREN, RIGHT_PAREN, COMMA, DOT, SEMICOLON,  PARAMETER,  EOF }</pre> |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

## 1.2. Token:

|                                                                                                                             |                                                                                                                                                                                                                      |
|-----------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>public record Token(     TokenType type,     String lexeme,     Object literal,     SourceLocation location ) {}</pre> | <p>Example:</p> <p>For: 123</p> <p>store:</p> <pre>type = INTEGER_LITERAL lexeme = "123" literal = 123</pre> <p>For: 'hello'</p> <p>store:</p> <pre>type = STRING_LITERAL lexeme = "'hello'" literal = "hello"</pre> |
|-----------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

## 1.3. Keyword Table:

```
public interface KeywordTable {
    Optional<TokenType> lookup(String text);
}

public final class StandardKeywordTable
    implements KeywordTable {

    private static final Map<String, TokenType> KEYWORDS =
        Map.ofEntries(
            Map.entry("SELECT", TokenType.SELECT),
            Map.entry("FROM", TokenType.FROM),
            Map.entry("WHERE", TokenType.WHERE),
            Map.entry("INSERT", TokenType.INSERT),
            Map.entry("UPDATE", TokenType.UPDATE),
            Map.entry("DELETE", TokenType.DELETE),
            Map.entry("GROUP", TokenType.GROUP),
            Map.entry("BY", TokenType.BY),
            Map.entry("ORDER", TokenType.ORDER),
            Map.entry("LIMIT", TokenType.LIMIT),
            Map.entry("OFFSET", TokenType.OFFSET),
            Map.entry("JOIN", TokenType.JOIN),
            Map.entry("LEFT", TokenType.LEFT),
            Map.entry("RIGHT", TokenType.RIGHT),
            Map.entry("INNER", TokenType.INNER),
            Map.entry("ON", TokenType.ON),
            Map.entry("AS", TokenType.AS),
            Map.entry("AND", TokenType.AND),
            Map.entry("OR", TokenType.OR),
            Map.entry("NOT", TokenType.NOT),
            Map.entry("NULL", TokenType.NULL)
        );

    @Override
    public Optional<TokenType> lookup(String text) {
        return Optional.ofNullable(
            KEYWORDS.get(text.toUpperCase())
        );
    }
}
```

### Lexer:

Input:

```
SELECT id, name
FROM users
WHERE age >= 18;
```

Output:

```
SELECT
IDENTIFIER(id)
COMMA
IDENTIFIER(name)
FROM
IDENTIFIER(users)
WHERE
IDENTIFIER(age)
GREATER_EQUAL
INTEGER_LITERAL(18)
SEMICOLON
EOF
```

```
public interface SqlLexer {
    List<Token> tokenize();
}
```

## 1.4. Implement Lexer

```
public final class SqlLexerImpl
    implements SqlLexer {
    private final String source;
    private final KeywordTable keywords;

    private int current;
    private int line = 1;
    private int column = 1;

    final List<Token> tokens = new ArrayList<>();

    SqlLexer(String source, KeywordTable keywords
    ) {
        this.source = source;
        this.keywords = keywords;
    }

    public List<Token> tokenize() {
        while (!isAtEnd()) scanToken();
        tokens.add(new Token(
            TokenType.EOF,
            "",
            null,
            new SourceLocation(current, line, column)
        ));

        return List.copyOf(tokens);
    }
}
```

Implement helper methods:

```
advance() | peek() | match() | add()
string()   | number() | identifier()
whitespace()
```

## 1.5. Comments

```
private void scanMinusOrComment() {
    if (match('-')) {
        while (!isAtEnd() && peek() != '\n') {
            advance();
        }
        return;
    }
    add(TokenType.MINUS);
}
```

```
private void scanToken() {
    char c = advance();

    switch (c) {
        case '(' -> add(TokenType.LEFT_PAREN);
        case ')' -> add(TokenType.RIGHT_PAREN);
        case ',' -> add(TokenType.COMMA);
        case '.' -> add(TokenType.DOT);
        case ';' -> add(TokenType.SEMICOLON);

        case '+' -> add(TokenType.PLUS);
        case '-' -> scanMinusOrComment();
        case '*' -> add(TokenType.STAR);
        case '/' -> scanSlashOrComment();
        case '%' -> add(TokenType.PERCENT);

        case '=' -> add(TokenType.EQUAL);

        case '<' -> add(
            match('=') ?
            TokenType.LESS_EQUAL :
            TokenType.LESS
        );

        case '>' -> add(
            match('=') ?
            TokenType.GREATER_EQUAL :
            TokenType.GREATER
        );

        case '"' -> string();

        case ' ', '\r', '\t', '\n' -> whitespace();

        default -> {
            if (isDigit(c)) {
                number();
            } else if (isIdentifierStart(c)) {
                identifier();
            } else {
                throw new IllegalArgumentException(
                    "Unexpected character: " + c
                );
            }
        }
    }
}
```

## 2. Grammer

### 2.1. Initial Grammar

sql

::= statement\* EOF

statement

::= selectStatement  
| insertStatement  
| updateStatement  
| deleteStatement  
| createTableStatement

### 2.2. SELECT Grammar

selectStatement

::= SELECT  
selectList  
fromClause?  
whereClause?  
groupByClause?  
havingClause?  
orderByClause?  
limitClause?  
offsetClause?  
", "  
,

selectList

::= "\*"   
| selectItem ("," selectItem)\*

selectItem

::= expression alias?

alias

::= AS IDENTIFIER  
| IDENTIFIER

### 2.3. FROM Grammar

fromClause

::= FROM tableReference

tableReference

::= tableFactor joinClause\*

tableFactor

::= tableName alias?  
| "(" selectStatement ")" alias?

tableName

::= qualifiedName

qualifiedName

::= IDENTIFIER ( "." IDENTIFIER )\*

### 2.4. Expression Grammar

expression

::= logicalOr

logicalOr

::= logicalAnd ("OR" logicalAnd)\*

logicalAnd

::= logicalNot ("AND" logicalNot)\*

logicalNot

::= NOT logicalNot  
| comparison

comparison

::= additive  
(  
comparisonOperator additive  
| IS NOT? NULL  
| IN "(" inBody ")"  
| BETWEEN additive AND additive  
| LIKE additive  
)\*

comparisonOperator

::= "=" | "<>" | "!="  
| "<" | "<=" | ">" | ">="

additive

::= multiplicative  
(("+" | "-") multiplicative)\*

multiplicative

::= unary  
(("\*" | "/" | "%") unary)\*

unary

::= ("+" | "-") unary  
| primary

primary

::= literal  
| qualifiedColumn  
| functionCall  
| "(" expression ")"  
| caseExpression  
| subqueryExpression  
| parameter

|                                                                                                                                                                                                                                                                                                          |                                                                                                                                                                                                                                                                                              |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p><b>2.5. JOIN Grammar</b></p> <p>joinClause<br/>       ::= joinType? JOIN tableFactor joinCondition?</p> <p>joinType<br/>       ::= INNER<br/>           LEFT OUTER?<br/>           RIGHT OUTER?<br/>           FULL OUTER?<br/>           CROSS</p> <p>joinCondition<br/>       ::= ON expression</p> | <p><b>2.7. ORDER / LIMIT / OFFSET</b></p> <p>orderByClause<br/>       ::= ORDER BY orderItem ("," orderItem)*</p> <p>orderItem<br/>       ::= expression (ASC   DESC)?</p> <p>limitClause<br/>       ::= LIMIT INTEGER_LITERAL</p> <p>offsetClause<br/>       ::= OFFSET INTEGER_LITERAL</p> |
| <p><b>2.6. WHERE / GROUP / HAVING</b></p> <p>whereClause<br/>       ::= WHERE expression</p> <p>groupByClause<br/>       ::= GROUP BY expressionList</p> <p>havingClause<br/>       ::= HAVING expression</p>                                                                                            | <p><b>2.8. Operator Precedence</b></p> <p>Input: a + b * c<br/>       must become:</p> $\begin{array}{c} + \\ / \ a \ * \\ / \ b \ c \end{array}$ <p>Therefore: a + (b * c)<br/>       is represented in the AST.</p>                                                                        |

## 3. AST

### 3.1. SqlNode:

```
public sealed interface SqlNode
    permits SqlStatement,
           Expression,
           TableReference,
           OrderByItem {

    SourceLocation location();
}
```

### 3.2. SqlStatement:

```
public sealed interface SqlStatement
    extends SqlNode
    permits SelectStatement,
           InsertStatement,
           UpdateStatement,
           DeleteStatement,
           CreateTableStatement {

}
```

### 3.3. SELECT AST

```
public record SelectStatement(
    boolean distinct,
    List<SelectItem> selectItems,
    TableReference from,
    Expression where,
    List<Expression> groupBy,
    Expression having,
    List<OrderByItem> orderBy,
    Integer limit,
    Integer offset,
    SourceLocation location
) implements SqlStatement {
}
```

### SELECT Item

```
public record SelectItem(
    Expression expression,
    String alias,
    SourceLocation location
) implements SqlNode {
}
```

Supports:

```
SELECT id
SELECT id AS user_id
SELECT COUNT(*) AS total
```

### 3.4. INSERT:

```
insertStatement
    ::= INSERT INTO tableName
        columnList?
        VALUES valueRows
```

```
valueRows
    ::= "(" expressionList ")"
       | "(" "(" expressionList ")" *
```

Example:  
 INSERT INTO users (id, name)  
 VALUES  
 (1, 'A'),  
 (2, 'B');

```
public record InsertStatement(
    TableName table,
    List<String> columns,
    List<List<Expression>> values,
    SourceLocation location
) implements SqlStatement {
}
```

Later support:

```
INSERT INTO users
SELECT ...
```

### 3.5. UPDATE:

```
UPDATE users
SET name = 'John',
    age = age + 1
WHERE id = 10;
```

```
public record UpdateStatement(
    TableName table,
    List<Assignment> assignments,
    Expression where,
    SourceLocation location
) implements SqlStatement {
}
```

### 3.6. DELETE:

```
DELETE FROM users
WHERE id = 10;
```

```
public record DeleteStatement(
    TableName table,
    Expression where,
    SourceLocation location
) implements SqlStatement {
}
```

### 3.7. CREATE TABLE:

```
createTableStatement
 ::= CREATE TABLE tableName
    "(" columnDefinition ("," columnDefinition)* ")"
```

```
columnDefinition
 ::= IDENTIFIER dataType columnConstraint*
```

```
dataType
 ::= INT
    | BIGINT
    | VARCHAR "(" INTEGER_LITERAL ")"
    | DECIMAL "(" INTEGER_LITERAL ","
INTEGER_LITERAL ")"
    | BOOLEAN
    | DATE
    | TIMESTAMP
```

```
public record CreateTableStatement(
    TableName table,
    List<ColumnDefinition> columns,
    SourceLocation location
) implements SqlStatement {
}
```

Example:

```
CREATE TABLE users (
  id INT,
  name VARCHAR(100),
  active BOOLEAN
);
```

### 3.8. TABLE AST

```
public sealed interface TableReference
    extends SqlNode
    permits TableName, Join, SubqueryTable {
}
```

```
public record TableName(
    QualifiedName name,
    String alias,
    SourceLocation location
) implements TableReference {
}

public record QualifiedName(List<String> parts) {}
```

### 3.9. JOIN AST

```
public enum JoinType {
    INNER, LEFT, RIGHT, FULL, CROSS
}
```

```
public record Join(
    TableReference left,
    JoinType type,
    TableReference right,
    Expression condition,
    SourceLocation location
) implements TableReference {
}
```

### 3.10. Column AST

```
public record ColumnReference(
    String qualifier,
    String name,
    SourceLocation location
) implements Expression {
}
```

For: "id" use:

- qualifier = null
- name = id

For: [u.id](#) use:

- qualifier = u
- name = id

For arbitrary qualification, use QualifiedName.

### 3.11. Literals

```
public sealed interface LiteralExpression
    extends Expression
    permits IntegerLiteral,
    DecimalLiteral,
    StringLiteral,
    BooleanLiteral,
    NullLiteral {
}
```

Example:

```
public record IntegerLiteral(
    long value,
    SourceLocation location
) implements LiteralExpression {
}
```

### 3.12. Binary Expression

```
public record BinaryExpression(
    Expression left,
    BinaryOperator operator,
    Expression right,
    SourceLocation location
) implements Expression {
}
```

```
public enum BinaryOperator {
    ADD, SUBTRACT,
    MULTIPLY, DIVIDE, MODULO,

    EQUAL, NOT_EQUAL,
    LESS, LESS_EQUAL,
    GREATER, GREATER_EQUAL,

    AND, OR,
    LIKE
}
```

### 7.14. NULL / IN/ BETWEEN

deleted\_at IS NULL

```
public record NullCheckExpression(
    Expression expression,
    boolean negated,
    SourceLocation location
) implements Expression {
}
```

**IN:**

id IN (1, 2, 3)

```
public record InExpression(
    Expression value,
    List<Expression> values,
    boolean negated,
    SourceLocation location
) implements Expression {
}
```

**BETWEEN:**

age BETWEEN 18 AND 60

```
public record BetweenExpression(
    Expression value,
    Expression lower,
    Expression upper,
    boolean negated,
    SourceLocation location
) implements Expression {
}
```

### 3.13. Unary Expression

```
public enum UnaryOperator {
    PLUS, MINUS, NOT
}
```

```
public record UnaryExpression(
    UnaryOperator operator,
    Expression expression,
    SourceLocation location
) implements Expression {
}
```

### 3.15. CASE

CASE

WHEN age >= 18 THEN 'adult'  
ELSE 'minor'

END

```
public record CaseExpression(
    List<WhenClause> clauses,
    Expression elseExpression,
    SourceLocation location
) implements Expression {
}
```

### 3.16. Function Calls

```
public record FunctionCallExpression(
    String name,
    List<Expression> arguments,
    boolean distinct,
    SourceLocation location
) implements Expression {
}
```

Examples:

COUNT(\*) | COUNT(DISTINCT user\_id)

SUM(amount | AVG(salary)

LOWER(name)

COALESCE(name, 'Unknown')

## 4. Parser

### Parser API:

```
public interface SqlParser {  
    List<SqlStatement> parse();  
}
```

### TokenStream:

Keep token navigation in one place.

```
public interface TokenStream {  
    Token peek();  
    Token peek(int distance);  
    Token previous();  
    Token advance();  
    boolean check(TokenType type);  
    boolean match(TokenType... types);  
    Token consume(  
        TokenType type,  
        String message  
    );  
    boolean isAtEnd();  
}
```

### Statement Dispatcher

As the parser grows, split:

- SelectParser
- ExpressionParser
- DmlParser
- DdlParser

```
private SqlStatement statement() {  
    return switch (tokens.peek().type()) {  
        case SELECT -> selectStatement();  
        case INSERT -> insertStatement();  
        case UPDATE -> updateStatement();  
        case DELETE -> deleteStatement();  
        case CREATE -> createStatement();  
  
        default -> throw error(  
            tokens.peek(),  
            "Expected SQL statement"  
        );  
    };  
}
```

## 4.1. SELECT Parser

```
private SelectStatement selectStatement() {  
    Token start = tokens.consume(TokenType.SELECT, "Expected SELECT");  
  
    boolean distinct = tokens.match(TokenType.DISTINCT);  
    List<SelectItem> items = selectList();  
  
    TableReference from = null;  
    if (tokens.match(TokenType.FROM)) from = fromClause();  
  
    Expression where = null;  
    if (tokens.match(TokenType.WHERE)) where = expression();  
  
    List<Expression> groupBy = List.of();  
    if (tokens.match(TokenType.GROUP)) {  
        tokens.consume(TokenType.BY, "Expected BY after GROUP");  
        groupBy = expressionList();  
    }  
  
    Expression having = null;  
    if (tokens.match(TokenType.HAVING)) having = expression();  
  
    List<OrderByItem> orderBy = List.of();  
    if (tokens.match(TokenType.ORDER)) {  
        tokens.consume(TokenType.BY, "Expected BY after ORDER");  
        orderBy = orderByList();  
    }  
}
```

```

Integer limit = null;
if (tokens.match(TokenType.LIMIT)) limit = parseInteger();

Integer offset = null;
if (tokens.match(TokenType.OFFSET)) offset = parseInteger();

return new SelectStatement(
    distinct,
    items,
    from,
    where,
    groupBy,
    having,
    orderBy,
    limit,
    offset,
    start.location()
);
}

```

## 4.2. Expression Parser

Start with recursive descent:

```

private Expression expression() {
    return logicalOr();
}

private Expression logicalOr() {
    Expression left = logicalAnd();

    while (tokens.match(TokenType.OR)) {
        Expression right = logicalAnd();

        left = new BinaryExpression(
            left,
            BinaryOperator.OR,
            right,
            left.location()
        );
    }

    return left;
}

```

```

private Expression logicalAnd() {
    Expression left = logicalNot();

    while (tokens.match(TokenType.AND)) {
        Expression right = logicalNot();

        left = new BinaryExpression(
            left,
            BinaryOperator.AND,
            right,
            left.location()
        );
    }

    return left;
}

```

## 4.3. Comparison Parser

```
private Expression comparison() {
    Expression left = additive();

    if (tokens.match(
        TokenType.EQUAL,
        TokenType.NOT_EQUAL,
        TokenType.LESS,
        TokenType.LESS_EQUAL,
        TokenType.GREATER,
        TokenType.GREATER_EQUAL
    )) {

        Token operator = tokens.previous();

        Expression right = additive();

        return new BinaryExpression(
            left,
            mapOperator(operator),
            right,
            operator.location()
        );
    }

    if (tokens.match(TokenType.IS)) {
        boolean negated =
tokens.match(TokenType.NOT);
        tokens.consume(TokenType.NULL,
"Expected NULL");

        return new NullCheckExpression(
            left,
            negated,
            left.location()
        );
    }

    // IN / BETWEEN / LIKE...

    return left;
}
```

Handle:

```
=
<>
!=
<
<=
>
>=
IS NULL
IS NOT NULL
IN
NOT IN
BETWEEN
NOT BETWEEN
LIKE
NOT LIKE
```

## 4.4. Pratt Parser Option

As expression syntax becomes large, a Pratt parser can replace the precedence methods.

**Example binding powers:**

|     |    |
|-----|----|
| OR  | 10 |
| AND | 20 |
| =   | 30 |
| <   | 30 |
| >   | 30 |
| +   | 40 |
| -   | 40 |
| *   | 50 |
| /   | 50 |
| %   | 50 |

**Recommended evolution:**

Initial implementation: **Recursive descent**  
Advanced implementation: **Pratt parser**

## 5. SQL Type System

Do not represent every SQL type as a string. The type model is required for: type checking, function resolution, implicit casts, NULL handling, expression validation, planner decisions

```
public sealed interface SqlType
    permits IntType,
           BigIntType,
           DecimalType,
           VarcharType,
           BooleanType,
           DateType,
           TimestampType,
           NullType {
}
```

Example:

```
public record DecimalType(
    int precision,
    int scale
) implements SqlType {
}
```

## 6. Semantic Analysis

Parsing answers: Is this syntactically valid?

Semantic analysis answers: Does this SQL make sense against a schema?

Example:

```
SELECT unknown_column
FROM users;
```

can parse correctly but fail semantic validation.

### 6.1. Catalog

**Possible implementations:**

InMemoryCatalog | JdbcCatalog | PostgresCatalog

MySqlCatalog | MockCatalog

```
public interface Catalog {
    Optional<TableMetadata> findTable(
        QualifiedName name
    );

    Optional<FunctionDefinition> findFunction(
        String name
    );
}
```

```
public record TableMetadata(
    QualifiedName name,
    List<ColumnMetadata> columns
) {}
```

```
public record ColumnMetadata(
    String name,
    SqlType type,
    boolean nullable
) {}
```

## 6.2. Scope Resolution

For:

```
SELECT u.id, o.total
FROM users u
JOIN orders o
  ON u.id = o.user_id;
```

Build: Query Scope

u -> users

o -> orders

Then resolve:

u.id -> users.id

o.total -> orders.total

## 6.3. Ambiguous Columns

If both tables contain id:

```
SELECT id
FROM users u
JOIN orders o
  ON u.id = o.user_id;
```

should produce:

error: column 'id' is ambiguous

while:

```
SELECT u.id
```

is valid.

This is semantic analysis, not parsing.

## 6.4. Function Registry

Do not hard-code every function into the parser.

```
public interface FunctionRegistry {
    Optional<FunctionDefinition> find(
        String name
    );
}
```

```
public record FunctionDefinition(
    String name,
    List<SqlType> parameterTypes,
    SqlType returnType,
    FunctionKind kind
) {}
```

```
public enum FunctionKind {
    SCALAR,
    AGGREGATE,
    WINDOW
}
```

Examples:

```
COUNT
SUM
AVG
MIN
MAX
LOWER
UPPER
COALESCE
SUBSTRING
```

## Subqueries

Support:

```
SELECT *  
FROM users  
WHERE id IN (  
    SELECT user_id  
    FROM orders  
);
```

Represent:

```
Select  
  Filter  
    InExpression  
      Subquery  
        Select
```

The parser should only build structure.

## CTEs

Support:

```
WITH active_users AS (  
    SELECT *  
    FROM users  
    WHERE active = true  
)  
SELECT *  
FROM active_users;
```

AST:

```
public record CommonTableExpression(  
    String name,  
    SelectStatement query,  
    SourceLocation location  
) implements SqlNode {  
}
```

## Set Operations

Support:

```
SELECT id FROM users  
UNION  
SELECT id FROM admins;
```

AST:

```
public record SetOperation(  
    SqlStatement left,  
    SetOperator operator,  
    boolean all,  
    SqlStatement right,  
    SourceLocation location  
) implements SqlStatement {  
}
```

```
public enum SetOperator {  
    UNION,  
    INTERSECT,  
    EXCEPT  
}
```

## Window Functions

Later support:

```
SELECT  
    name,  
    department_id,  
    ROW_NUMBER() OVER (  
        PARTITION BY department_id  
        ORDER BY salary DESC  
    ) AS rank  
FROM employees;
```

AST:

```
public record WindowExpression(  
    Expression function,  
    List<Expression> partitionBy,  
    List<OrderByItem> orderBy,  
    WindowFrame frame,  
    SourceLocation location  
) implements Expression {  
}
```

## SQL Dialects

SQL dialects differ:

PostgreSQL / MySQL / SQLite / Oracle  
SQL Server

Avoid scattering:

if (postgres) ...  
if (mysql) ...  
if (oracle) ...

Use:

```
public interface SqlDialect {  
    KeywordTable keywords();  
    IdentifierRules identifiers();  
    GrammarFeatures features();  
}
```

### Dialect Features

```
public record GrammarFeatures(  
    boolean supportsLimit,  
    boolean supportsOffset,  
    boolean supportsReturning,  
    boolean supportsRecursiveCte,  
    boolean supportsWindowFunctions  
) {}
```

Implement:

```
StandardSqlDialect  
PostgresDialect  
MySqlDialect  
SqliteDialect
```

## Logical Query Processing

A planner can model the relational  
operations approximately as:

```
FROM  
↓  
WHERE  
↓  
GROUP BY  
↓  
HAVING  
↓  
SELECT  
↓  
ORDER BY  
↓  
LIMIT
```

This is why a logical plan is  
useful.

### Logical Plan

```
public sealed interface LogicalPlan  
    permits LogicalScan,  
             LogicalFilter,  
             LogicalProject,  
             LogicalJoin,  
             LogicalAggregate,  
             LogicalSort,  
             LogicalLimit {  
}
```

```
public record LogicalScan(  
    TableMetadata table  
) implements LogicalPlan {}
```

```
public record LogicalFilter(  
    LogicalPlan input,  
    Expression predicate  
) implements LogicalPlan {}
```

```
public record LogicalProject(  
    LogicalPlan input,  
    List<Expression> expressions  
) implements LogicalPlan {}
```

| Example Plan                                                                                                                                                                     | JOIN Plan                                                                                                                                        |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>SELECT name FROM users WHERE age &gt;= 18;</pre>                                                                                                                            | <pre>SELECT u.name, o.total FROM users u JOIN orders o ON u.id = o.user_id;</pre>                                                                |
| <p><b>Logical plan:</b></p> <pre>Project(name)     Filter(age &gt;= 18)     Scan(users)</pre> <p>The parser creates the AST;<br/>the planner creates this<br/>representation</p> | <p><b>Logical plan:</b></p> <pre>Project(u.name, o.total)     Join  /      Scan(users) Scan(orders)</pre> <p>condition:<br/>u.id = o.user_id</p> |

## Optimizer:

```
public interface OptimizationRule {
    boolean matches(LogicalPlan plan);
    LogicalPlan apply(LogicalPlan plan);
}

public final class PlanOptimizer {
    private final List<OptimizationRule> rules;

    public PlanOptimizer(List<OptimizationRule> rules) {
        this.rules = List.copyOf(rules);
    }

    public LogicalPlan optimize(LogicalPlan plan) {
        LogicalPlan current = plan;
        boolean changed;

        do {
            changed = false;
            for (OptimizationRule rule : rules) {
                if (rule.matches(current)) {
                    LogicalPlan next = rule.apply(current);

                    if (next != current) {
                        current = next;
                        changed = true;
                    }
                }
            }
        } while (changed);
        return current;
    }
}
```

## Execution Engine

Optional future phase:

```
public interface PhysicalOperator {  
    void open();  
    Row next();  
    void close();  
}
```

Operators:

TableScan  
IndexScan  
Filter  
Project  
HashJoin  
NestedLoopJoin  
Aggregate  
Sort  
Limit

## Row Model

Start simple:

```
public record Row(  
    List<Object> values  
) {}
```

Later consider:

columnar execution  
vectorized execution  
primitive arrays  
off-heap memory

## Storage Boundary

pages  
B+ trees  
buffer pools  
WAL  
disk offsets  
MVCC

Future architecture:

SQL Parser



Planner



Executor



Storage Engine



+- Buffer Pool

+- Heap Files

+- B+ Tree

+- WAL

+- Transactions

## Parser Configuration

```
public record ParserConfiguration(  
    SqlDialect dialect,  
    boolean collectDiagnostics,  
    boolean allowMultipleStatements,  
    boolean allowExtensions  
) {}
```

## Top-Level Compilation API

```
public final class SqlCompiler {
```

```
    public QueryCompilationResult compile(String sql) {
        List<Token> tokens = lexer.tokenize(sql);
        List<SqlStatement> ast = parser.parse(tokens);
        ValidatedSql validated = semanticAnalyzer.analyze(ast);
        LogicalPlan logical = planner.plan(validated);
        LogicalPlan optimized = optimizer.optimize(logical);
        PhysicalPlan physical = physicalPlanner.plan(optimized);

        return new QueryCompilationResult(
            ast,
            validated,
            logical,
            optimized,
            physical
        );
    }
}
```

Conceptually:

```
SQL
↓
Tokens
↓
AST
↓
Validated AST
↓
Logical Plan
↓
Optimized Logical Plan
↓
Physical Plan
```

## Dependency Injection

```
public final class SqlCompiler {
```

```
    private final SqlLexer lexer;
    private final SqlParser parser;
    private final SemanticAnalyzer semanticAnalyzer;
    private final QueryPlanner planner;
    private final PlanOptimizer optimizer;
```

```
    public SqlCompiler(
        SqlLexer lexer,
        SqlParser parser,
        SemanticAnalyzer semanticAnalyzer,
        QueryPlanner planner,
        PlanOptimizer optimizer
    ) {
        this.lexer = lexer;
        this.parser = parser;
        this.semanticAnalyzer = semanticAnalyzer;
        this.planner = planner;
        this.optimizer = optimizer;
    }
}
```

Conceptually:

```
SQL
↓
Tokens
↓
AST
↓
Validated AST
↓
Logical Plan
↓
Optimized Logical Plan
↓
Physical Plan
```